Customer Churn Prediction Using Machine Learning

Mac Tanatswa Muza | 12405348

Department of Computer Science and Engineering | Lovely Professional University
Python Machine Learning Internship

muzatanatswa037@gmail.com

Abstract—Customer churn is an important problem for subscription-based businesses because losing an existing customer can affect recurring revenue and requires the company to find a replacement. This project applies supervised machine learning to the Telco Customer Churn dataset and compares three classification algorithms: Logistic Regression, K-Nearest Neighbors (KNN), and Gaussian Naive Bayes. The supplied CSV contains 7,043 customer records and 21 columns. In the implemented notebook, blank values were first converted to missing values, TotalCharges was converted to numeric form and its missing values were filled with the median, and customerID was removed. Object-type columns were encoded with LabelEncoder, the target Churn was separated from the features, and StandardScaler was applied before an 80/20 train-test split with random_state=42. The models were evaluated using prediction accuracy on the held-out test set. Logistic Regression achieved 81.55% accuracy, compared with 75.73% for KNN and 75.80% for Gaussian Naive Bayes. Among the three models, Logistic Regression produced the highest test accuracy in this experiment. The result shows that a simple classification model can provide a useful baseline for churn prediction, while also highlighting the importance of comparing different algorithms on the same dataset. The reported values should be interpreted as the results of the implemented notebook, including its preprocessing order.

Keywords— Customer Churn, Machine Learning, Logistic Regression, K-Nearest Neighbors, Naive Bayes, Classification, Python

I. INTRODUCTION

Customer churn refers to a customer stopping business with a company, such as cancelling a subscription or switching to another provider. In telecommunications, churn is important because customers usually use services over an extended period, so retaining existing customers can be valuable. Predicting churn before it happens can help a company identify customers who may need additional attention, support, or retention offers. The project material explains that machine learning can learn patterns from historical customer information and use those patterns to classify customers according to their churn outcome. [1]

The purpose of this project was to apply that idea to an actual customer dataset rather than only studying the algorithms theoretically. The notebook loads the Telco Customer Churn CSV, performs a sequence of preprocessing steps, trains three classification models, generates predictions for a held-out test set, and compares the resulting accuracy values. The project therefore follows a straightforward supervised-learning workflow: data preparation, feature transformation, model training, prediction, and evaluation.

The comparison is intentionally limited to the three algorithms implemented in the notebook. The aim is not to claim that one algorithm is always best for churn prediction. Instead, the experiment asks which of the tested models achieved the highest accuracy on this particular dataset using this particular implementation.

II. PROJECT OBJECTIVES

The project has four connected objectives. First, it aims to predict whether a customer will churn by training classification models on historical customer information. Second, it compares Logistic Regression, KNN, and Naive Bayes using the same processed dataset. Third, it evaluates the predictions using accuracy as the primary metric. Finally, it identifies the model with the highest accuracy in this experiment. These objectives match the supplied project material, which presents model comparison as an important part of selecting a suitable algorithm for the specific problem rather than assuming that one method is universally superior. [1]

III. DATASET DESCRIPTION

The supplied Telco Customer Churn CSV contains 7,043 rows and 21 columns. The target variable is `Churn`, with two classes: `Yes` and `No`. The dataset contains customer identifiers, demographic and account information, subscribed services, contract information, billing information, and the churn label. The CSV contains 11 blank-string entries in `TotalCharges`; these are handled explicitly in the notebook. Apart from those blank strings, the raw CSV does not contain pandas-recognized null values.

The original columns include customerID, gender, SeniorCitizen, Partner, Dependents, tenure, PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges, and Churn. After customerID is removed and Churn is separated as the target, the model receives 19 predictor features. Four predictors are numeric after conversion (SeniorCitizen, tenure, MonthlyCharges, and TotalCharges), while 15 predictors originate as object/categorical columns and are encoded with LabelEncoder.

Variable Group	Examples from Supplied CSV
Customer / account	gender, SeniorCitizen, Partner, Dependents, tenure
Service	PhoneService, MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport
Streaming	StreamingTV, StreamingMovies
Contract / billing	Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges
Target	Churn

TABLE I
MAJOR FEATURE GROUPS IN THE DATASET

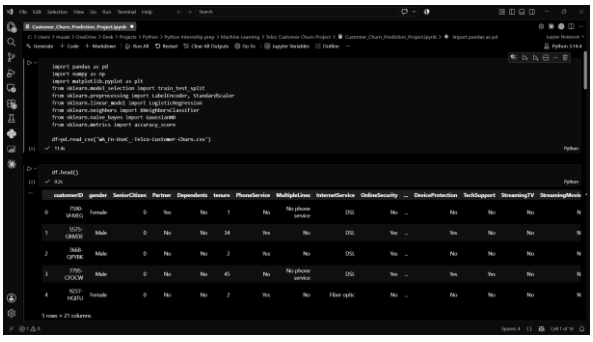
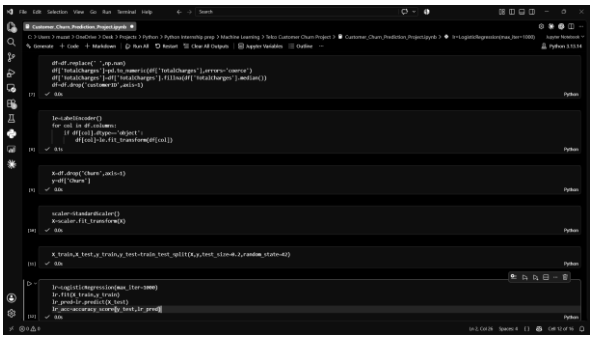


Fig. 2. Dataset loaded and inspected in the Jupyter Notebook.

Dataset property	Verified value
Rows	7,043
Columns	21

Dataset property	Verified value
Target	Churn (Yes / No)
Blank TotalCharges values	11
Predictors after removing customerID and Churn	19
Training / test split	5,634 / 1,409

TABLE II
VERIFIED DATASET AND SPLIT SUMMARY



IV. DATA PREPROCESSING

The preprocessing follows the order visible in the notebook. First, the code replaces the literal blank string with `np.nan` across the dataframe. It then converts `TotalCharges` to numeric using `pd.to_numeric(..., errors='coerce')`. Any resulting missing values in `TotalCharges` are filled with the column median. In the supplied CSV, the median of the non-blank numeric TotalCharges values is 1397.475. The `customerID` column is then removed because it is an identifier rather than a customer characteristic intended for prediction.

Next, the notebook creates a `LabelEncoder` and applies `fit_transform` to every object-type column. This converts the categorical values into numeric labels so that the scikit-learn classifiers can receive a numeric feature matrix. The code does not use one-hot encoding.

The feature matrix `X` is created by dropping `Churn`, while `y` contains the `Churn` column. The notebook then creates a `StandardScaler` and applies `fit_transform` to the complete feature matrix. Standardization puts features on a common scale, which is particularly relevant to distance-based methods such as KNN. The notebook then performs `train_test_split` with `test_size=0.2` and `random_state=42`, producing 5,634 training rows and 1,409 test rows. [2]

One implementation detail is important when interpreting the results: the scaler is fitted before the train-test split. Therefore, the scaling statistics are calculated using the complete dataset rather than only the training portion. This is faithfully documented here because the report is intended to describe the actual implementation.

Selected preprocessing code

```
df=df.replace(' ',np.nan)
df['TotalCharges']=pd.to_numeric(df['TotalCharges'],errors=
'coerce')
df['TotalCharges']=df['TotalCharges'].fillna(df['TotalCharg
es'].median())
df=df.drop('customerID',axis=1)

le=LabelEncoder()
for col in df.columns:
    if df[col].dtype=='object':
        df[col]=le.fit_transform(df[col])
```

Fig. 3. Preprocessing, feature-target separation, scaling, train-test split, and Logistic Regression implementation.

V. MACHINE LEARNING MODELS

Logistic Regression

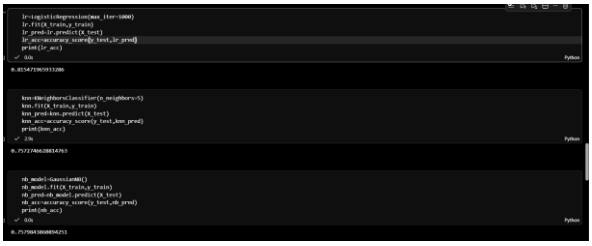
Logistic Regression is used as a binary classification model for the churn target. The notebook initializes it with `LogisticRegression(max_iter=1000)`, fits it on the training data, predicts the test labels, and evaluates those predictions with `accuracy_score`. The `max_iter=1000` setting gives the solver a larger iteration limit than the library default and is the parameter actually used in the project. [3]

K-Nearest Neighbors (KNN)

KNN classifies a new sample using the classes of nearby training samples. The notebook uses `KNeighborsClassifier(n_neighbors=5)`, so five nearest neighbors are considered when making a prediction. The model is fitted on the training set and then used to predict the test set. Because KNN is based on distances between observations, the StandardScaler step is relevant to this model. [4]

Gaussian Naive Bayes

Gaussian Naive Bayes is a probabilistic classification method based on Bayes' theorem and a conditional independence assumption. The notebook uses the default `GaussianNB()` implementation, trains it on the same training data as the other models, and evaluates its predictions with the same accuracy measure. [5]



VI. IMPLEMENTATION

The implementation is contained in a Jupyter Notebook written in Python. The imports visible in the notebook are pandas, NumPy, matplotlib, and selected components from scikit-learn. Pandas is used for reading and manipulating the CSV data, NumPy is used for the missing-value replacement, matplotlib is used for visualization, and scikit-learn supplies the data split, encoding, scaling, three classifiers, and accuracy metric.

The overall implementation can be summarized by the following sequence:
1) load the CSV; 2) inspect the data using `head`, `shape`, `info`, `describe`, and `isnull`; 3) clean `TotalCharges` and remove `customerID`; 4) encode object columns; 5) separate `X` and `y`; 6) scale the features; 7) split the data; 8) train each classifier; 9) predict the test labels; 10) calculate accuracy; and 11) create a comparison dataframe and bar chart.

Fig. 2 shows the notebook loading the CSV and displaying the first five rows. Fig. 3 shows the main preprocessing steps and the Logistic Regression implementation. Fig. 4 shows the three model prediction and accuracy calculations. These screenshots are included as evidence of the actual implementation rather than as substitutes for the written explanation.

VII. EXPERIMENTAL RESULTS

The three models were evaluated on the same test set using `accuracy_score`. Table III reports the exact values rounded to two decimal places. Logistic Regression achieved 81.55%, KNN achieved 75.73%, and Gaussian Naive Bayes achieved 75.80%. The full-

precision values printed by the notebook were 0.815471965933286, 0.757274662881476, and 0.757984386089425, respectively.

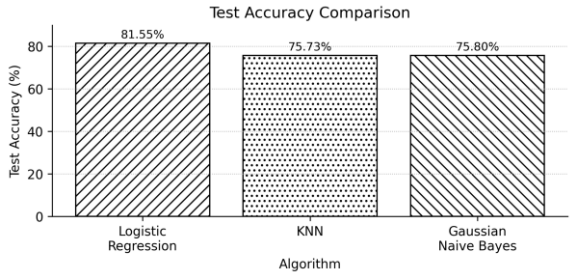
Logistic Regression was the highest-performing model by test accuracy. It exceeded KNN by approximately 5.82 percentage points and Gaussian Naive Bayes by approximately 5.75 percentage points. The result table and the bar chart were generated directly from the three stored accuracy variables in the notebook. Fig. 4 provides the raw printed accuracy outputs, while Fig. 1 provides the clean accuracy comparison chart.

Model	Test Accuracy
Logistic Regression	81.55%
KNN	75.73%
Gaussian Naive Bayes	75.80%

TABLE III
MODEL PERFORMANCE COMPARISON

Fig. 4. Actual model training, predictions, and printed test-accuracy values.

VIII. RESULT VISUALIZATION



IX. DISCUSSION

The main result is that Logistic Regression performed better than the two other tested models under the exact preprocessing and train-test procedure used in the notebook. A reasonable interpretation is that the transformed feature representation provided a useful setting for a linear classifier. Logistic Regression can combine the standardized input variables into a decision function for the binary churn outcome, which may have matched the structure of this dataset better than the local-distance approach used by KNN.

KNN produced the lowest accuracy of the three models. KNN depends on distances between observations, so the way categorical variables were converted into integer labels can affect those distances. This does not prove that KNN is unsuitable for churn prediction; it only describes a plausible reason why this particular implementation may have performed less well. The use of five neighbors was also fixed rather than tuned across several possible values.

Gaussian Naive Bayes produced an accuracy of 75.80%, almost the same as KNN at 75.73%. The two results are close, but the notebook does not calculate additional diagnostics that would explain the types of errors made by either model. Therefore, the strongest supported conclusion is simply that both were below Logistic Regression on the reported test-accuracy measure.

The preprocessing also matters. Converting the categorical columns into numeric values allows the models to operate on the data, and standardization places the input variables on a common scale. The scikit-learn documentation describes StandardScaler as a method for centering features and scaling them to unit variance. [2] However, because the scaler was fitted before the split in the actual notebook, the evaluation is not a strictly leakage-free estimate of generalization performance. This is discussed as an implementation limitation rather than silently changing the project result.

X. LIMITATIONS

There are several limitations in the completed experiment. First, only three classification algorithms were tested, so the comparison is narrow. Second, accuracy was the primary and only reported evaluation metric. The dataset contains 5,174 non-churned customers and 1,869 churned customers, so the class distribution is not balanced; accuracy alone therefore does not show how well the models detect the churn class. Third, the notebook does not report precision, recall, F1-score, ROC-AUC, or a confusion matrix. Fourth, the hyperparameters were mostly left at the selected/default values, with `max_iter=1000` for Logistic Regression and `n_neighbors=5` for KNN, rather than being systematically tuned.

Finally, the notebook fits StandardScaler on the complete feature matrix before the train-test split. This means the test data contributes to the scaling statistics. The result is still the actual output of the completed project, but this ordering should be considered when interpreting the reported accuracy. The project also does not include a deployed production application.

XI. FUTURE WORK

Future work can improve the experiment without changing what was actually completed. Additional classification algorithms could be tested and their hyperparameters tuned using cross-validation. A confusion matrix and precision, recall, F1-score, and ROC-AUC would provide a more complete view of churn-class performance. Class imbalance could also be examined more carefully, especially if the main business goal is to identify customers who are likely to leave. Alternative categorical encoding methods could be compared with the current LabelEncoder approach. Finally, the best-performing model could be placed behind a simple web interface to demonstrate how a churn prediction could be used outside the notebook. These are proposed improvements, not completed parts of the present project.

XII. CONCLUSION

This project demonstrates a complete introductory machine-learning workflow for customer churn classification. The Telco Customer Churn dataset was cleaned, categorical values were encoded, features were standardized, and the data was divided into training and test sets. Logistic Regression, KNN, and Gaussian Naive Bayes were then trained and evaluated using test accuracy. Logistic Regression achieved the highest result at 81.55%, followed by Gaussian Naive Bayes at 75.80% and KNN at 75.73%.

The result supports selecting Logistic Regression as the best-performing model among the three tested models for this particular experiment. It does not establish that Logistic Regression is universally the best approach to customer churn. A broader evaluation using additional metrics, better-controlled preprocessing, more algorithms, and cross-validation would be needed before making a stronger model-selection claim.

XIII. REFERENCES

[1] IBM, “Telco Customer Churn Dataset,” Kaggle.
[2] scikit-learn developers, “StandardScaler,” scikit-learn documentation.
[3] scikit-learn developers, “LogisticRegression,” scikit-learn documentation.
[4] scikit-learn developers, “KNeighborsClassifier,” scikit-learn documentation.
[5] scikit-learn developers, “GaussianNB,” scikit-learn documentation.